AC1 | Teste exemplo — resolução completa

Guia de consulta: código MIPS dos exercícios 1, 2 e 3, análise do programa do exercício 4 e explicação das 10 respostas. Base: ac1_teste_exemplo.pdf (5 páginas). Os códigos usam as syscalls do MARS e mantêm a sequência lógica do C apresentado.

Nota: Este ficheiro é uma resolução comentada, não uma transcrição de ficheiros f11.s/f12.s/f13.s fornecidos pela disciplina. O próprio enunciado refere que o teste exemplo não é um teste-modelo e exige comentários em linguagem C.

Exercício 1 (5 valores) — máximo e mínimo

Ler até 20 inteiros (ou até aparecer zero), ignorar zero na atualização de máximo/mínimo e imprimir o resultado no formato Maximo/minimo sao: max:min.

Mapa de registos

Registo	Variável	Inicialização
$t0	n	0
$t1	min	0x7FFFFFFF
$t2	max	0x80000000
$t3	val	obtido com read_int

```
 .data
msg1: .asciiz "Digite ate 20 inteiros (zero para terminar):"
msg2: .asciiz "Maximo/minimo sao: "

.text
.globl main
main:
    li $t0,0              # n = 0
    li $t1,0x7FFFFFFF     # min = INT_MAX
    li $t2,0x80000000     # max = INT_MIN
    la $a0,msg1           # print_string(...)
    li $v0,4
    syscall

do:
    li $v0,5              # val = read_int()
    syscall
    move $t3,$v0
    beq $t3,$zero,endif   # if (val != 0)
    ble $t3,$t2,endifmax  # if (val > max)
    move $t2,$t3          # max = val
endifmax:
    bge $t3,$t1,endifmin  # if (val < min)
    move $t1,$t3          # min = val
endifmin:
endif:
    addiu $t0,$t0,1       # n++
    bge $t0,20,enddo      # while (n < 20 && val != 0)
    bne $t3,$zero,do
enddo:
    la $a0,msg2           # print_string(...)
    li $v0,4
    syscall
    move $a0,$t2          # print_int10(max)
    li $v0,1
    syscall
    li $a0,':'            # print_char(':')
    li $v0,11
```

```
    syscall
    move $a0,$t1          # print_int10(min)
    li $v0,1
    syscall
    jr $ra
```

Raciocínio passo a passo

1. Inicializar min com o maior inteiro com sinal e max com o menor garante que o primeiro valor não nulo os pode atualizar. 2. A mensagem é emitida uma vez, antes do ciclo. 3. A syscall 5 devolve o inteiro em $v0; copiamos para $t3. 4. Se val = 0, saltamos apenas o bloco de atualização. 5. As condições de salto são o inverso das condições dos if do C. 6. O incremento n++ acontece sempre, inclusive quando val é zero. 7. Repetimos apenas quando n ainda é inferior a 20 e val não é zero.

Teste: entrada 8, -3, 15, 2, 0 → saída final Maximo/minimo sao: 15:-3. Se a primeira entrada for zero, min e max conservam os valores iniciais, tal como no C fornecido.

Exercício 2 (5 valores) — trocar metades de um array

SIZE = 8; trocar val[0] com val[4], val[1] com val[5], val[2] com val[6] e val[3] com val[7]. Depois imprimir todos os elementos com ponteiro p.

Mapa de registos

Registo	O que guarda
$t0	i
$t1	v = antigo val[i]
$t2	endereço inicial de val
$t3	p (ponteiro na impressão)
$t4	i * 4
$t5	endereço de val[i]
$t6	valor de val[i+4]
$t7	val + SIZE (endereço final)

```
.eqv SIZE,8
.data
val: .word 8,4,15,-1987,327,-9,27,16
msg: .asciiz "Result is: "

.text
.globl main
main:
    li $t0,0             # i = 0
    la $t2,val           # &(val[0])
do:
    sll $t4,$t0,2        # i * 4 (bytes)
    addu $t5,$t2,$t4     # &val[i]
    lw $t1,0($t5)        # v = val[i]
    lw $t6,16($t5)       # ler val[i + SIZE/2]
    sw $t6,0($t5)        # val[i] = val[i + SIZE/2]
    sw $t1,16($t5)       # val[i + SIZE/2] = v
    addiu $t0,$t0,1      # ++i
    blt $t0,4,do         # while (++i < SIZE/2)

    la $a0,msg           # print_string("Result is: ")
    li $v0,4
    syscall
    move $t3,$t2         # p = val
    addiu $t7,$t2,32     # val + SIZE (8 * 4 bytes)
while:
    bgeu $t3,$t7,endw    # while (p < val + SIZE)
    lw $a0,0($t3)        # *p
    li $v0,1             # print_int10(*p++)
    syscall
    addiu $t3,$t3,4      # p++
    li $a0,','           # print_char(',')
    li $v0,11
    syscall
    j while
endw:
    jr $ra
```

Porque 16($t5)?

O registo $t5 aponta para val[i]. O elemento val[i + SIZE/2] está quatro inteiros à frente: SIZE/2 = 4, portanto 4 × 4 bytes = 16 bytes. lw lê; os dois sw trocam os valores na memória. Usar move aqui não trocaria os elementos do array.

Porque o ponteiro avança 4?

p é um int*: aponta para um inteiro de 4 bytes. Em C, *p++ significa ler o valor apontado e, em seguida, avançar o ponteiro. No MIPS: lw $a0,0($t3), syscall de impressão e addiu $t3,$t3,4. O limite val + SIZE é um endereço 32 bytes depois do início, não 8 bytes.

Resultado: Result is: 327,-9,27,16,8,4,15,-1987, (com vírgula final, como no código C).

Exercício 3 (5 valores) — separar inteiros ímpares

Ler 35 inteiros para a, percorrer a com p1, copiar apenas os ímpares para b com p2, contar pares/ímpares e imprimir os elementos copiados para b.

Mapa de registos

Registo	O que guarda
$t0	n_even
$t1	n_odd
$t2	p1
$t3	p2
$t4	início de a
$t5	início de b
$t6	a + N (limite final)
$t7	*p1 (valor atual)
$t8	bit de paridade / n_odd * 4
$t9	b + n_odd (limite final)

```
.eqv N,35
.data
a: .space 140
b: .space 140

.text
.globl main
main:
    li $t0,0             # n_even = 0
    li $t1,0             # n_odd = 0
    la $t4,a             # inicio de a
    la $t5,b             # inicio de b
    addiu $t6,$t4,140    # a + N

    move $t2,$t4         # p1 = a
for_read:
    bgeu $t2,$t6,end_read # p1 < a + N
    li $v0,5             # *p1 = read_int()
    syscall
    sw $v0,0($t2)
    addiu $t2,$t2,4      # p1++
    j for_read
end_read:
    move $t2,$t4         # p1 = a
    move $t3,$t5         # p2 = b
for_sep:
    bgeu $t2,$t6,end_sep # p1 < a + N
    lw $t7,0($t2)        # *p1
    andi $t8,$t7,1       # (*p1 % 2) != 0
    beq $t8,$zero,else   # se par, ir a else
    sw $t7,0($t3)        # *p2 = *p1
    addiu $t3,$t3,4      # p2++ (so nos impares)
    addiu $t1,$t1,1      # n_odd++
    j endif
else:
    addiu $t0,$t0,1      # n_even++
endif:
    addiu $t2,$t2,4      # p1++ (sempre)
    j for_sep
end_sep:
```

```
    move $t3,$t5         # p2 = b
    sll $t8,$t1,2        # n_odd * 4
    addu $t9,$t5,$t8     # b + n_odd
for_print:
    bgeu $t3,$t9,end_print # p2 < b + n_odd
    lw $a0,0($t3)        # *p2
    li $v0,1             # print_int10(*p2)
    syscall
    addiu $t3,$t3,4      # p2++
    j for_print
end_print:
    jr $ra
```

O detalhe que decide o exercício

p1++ acontece sempre, depois do if/else; p2++ acontece só se o valor for ímpar. Se avançasses p2 também para valores pares, deixarias buracos no array b. Em MIPS um int* avança com addiu ...,4. Para verificar ímpar, andi com 1 observa o bit menos significativo (funciona também para negativos).

O terceiro ciclo termina em b + n_odd: calcula-se n_odd * 4 com sll ...,2 e soma-se ao endereço inicial de b. Apenas os números efetivamente copiados são impressos, sem separadores adicionais (o C fornecido não os imprime).

Exercício 4 (5 valores) — análise de código MIPS

Neste exercício não se escreve código novo. As perguntas usam as etiquetas de dados, endereços dos registos e os efeitos do ciclo. O MARS permite verificar registos, Data Segment e Labels.

```
.data
X1:    .asciiz "EX-T1"
       .align 2
X2:    .space 20
X3:
.text
.globl main
main:  la   $t4,X2
       ori  $t5,$0,4
       xor  $t0,$t0,$t0
       xor  $t1,$t1,$t1
L1:    beq  $t0,$t5,L2
       add  $t2,$t0,$t0
       add  $t3,$t2,$t2
       addu $t3,$t3,$t4
       sw   $t2,0($t3)
       add  $t1,$t1,$t2
       addi $t0,$t0,1
       j    L1
L2:    sw   $t1,4($t3)
       jr   $ra
```

Antes de responder: mapa de memória e execução

X1 começa em 0x10010000. A string EX-T1 usa 5 caracteres + terminador zero = 6 bytes. O alinhamento .align 2 avança a próxima declaração para um endereço múltiplo de 4: X2 começa em 0x10010008. X2 reserva 20 bytes e X3 marca 0x1001001C (28 bytes depois de X1).

t0 ao entrar no corpo	t2 = 2*t0	t3 = X2 + 4*t0	t1 após somar t2
0	0	X2 + 0	0
1	2	X2 + 4	2
2	4	X2 + 8	6
3	6	X2 + 12	12

Quando t0 chega a 4, a instrução beq sai para L2. No fim: t1 = 12 = 0xC; t3 = X2 + 12 = 0x10010014; a instrução sw $t1,4($t3) escreve no endereço 0x10010018.

Resolução das 10 perguntas, passo a passo

Pergunta 1 — Endereço de X2[3] | Resposta C

X2 = 0x10010008; índice 3 significa deslocamento 3 × 4 = 12 = 0xC. Logo 0x10010008 + 0xC = 0x10010014.

Pergunta 2 — Endereço de X3 | Resposta D

X1 ocupa 6 bytes. .align 2 coloca X2 em 0x10010008. .space 20 = 0x14 bytes. Assim X3 = 0x10010008 + 0x14 = 0x1001001C.

Pergunta 3 — Dimensão de X2 como array de int | Resposta A

X2 reserva 20 bytes; cada int ocupa 4 bytes: 20 / 4 = 5 inteiros.

Pergunta 4 — Memória de dados X3 - X1 | Resposta A

0x1001001C - 0x10010000 = 0x1C. Em decimal: 1 × 16 + 12 = 28 bytes.

Pergunta 5 — Valor final de $t1 | Resposta C

t0 assume 0, 1, 2, 3 no corpo. Em cada passagem t2 = 2*t0: 0, 2, 4, 6. t1 acumula 0+2+4+6 = 12 = 0x0000000C.

Pergunta 6 — Endereço do último sw | Resposta C

Depois do ciclo, t3 = X2 + 3*4 = 0x10010014. O deslocamento 4($t3) soma 4 bytes: 0x10010018.

Pergunta 7 — Número de execuções completas | Resposta A

O corpo corre quando t0 = 0, 1, 2, 3: 4 execuções. Quando t0 = 4, beq salta antes do corpo.

Pergunta 8 — Valor de X2[3] | Resposta D

Na iteração t0 = 3, t2 = t0 + t0 = 6. sw $t2,0($t3) escreve o 6 em X2[3], ou 0x00000006.

Pergunta 9 — Endereço da etiqueta L2 | Resposta A

O painel Labels do MARS mostra L2 = 0x0040008C no ambiente do exemplo (início do código 0x00400058). A expansão de pseudo-instruções, nomeadamente la, altera a contagem de instruções reais.

Pergunta 10 — Bytes da string X1 | Resposta D

"EX-T1" tem 5 caracteres; .asciiz acrescenta um byte zero (\0). Total: 6 bytes. Não confundir com os 8 bytes antes de X2, que incluem 2 bytes de alinhamento.

Grelha rápida de respostas

Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8	Q9	Q10
C	D	A	A	C	C	A	D	A	D

Erros a evitar na última revisão

la / lw / sw: la obtém endereço; lw lê o conteúdo da memória; sw escreve na memória. Índice / ponteiro: i++ significa índice +1; p++ com int* significa endereço +4. Endereço / valor: se um registo contém o valor do elemento, não o uses como base em 0($registo). if: salta para endif quando a condição em C é falsa. for: incrementa antes de saltar de volta. do-while: testa no final. Hexadecimal: 0x1C = 28 decimal; o byte seguinte a 0x1C é 0x1D, mas o inteiro seguinte alinhado está em 0x20.

Verificação no MARS: Assemble → Execute → Run; consulta Labels para endereços de etiquetas, Registers para valores finais, Data Segment para palavras guardadas e Text Segment para o endereço das instruções reais. Para acompanhar iterativamente, usa Step ou breakpoints.